Proyecto Final. Informe de Avance 3: Detección y Mitigación de Daños Frente a Ataques de Ransomware — Prueba de Concepto

Ivo Gómez Enrico
Departamento Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional, Facultad Regional Santa Fe
Santa Fe, Argentina — igomezenrico@frsf.utn.edu.ar

Francisco Kuchen
Departamento Ingeniería en Sistemas de Información
Universidad Tecnológica Nacional, Facultad Regional Santa Fe
Santa Fe, Argentina — fkuchen@frsf.utn.edu.ar

Abstract—Este informe presenta el tercer avance del Proyecto Final: el prototipo operativo de los módulos de detección y recuperación de Guardian FS validado sobre el conjunto base de pruebas. Se completó la campaña de recolección y el entrenamiento del modelo de Machine Learning con la metodología sin fuga de información (1805 ventanas, 127 sesiones), incorporando las clases benignas de alta entropía cuya ausencia producía falsos positivos reales. La campaña experimental sobre los tres niveles del diseño (sin protección / ZFS solo / ZFS + detección activa) muestra que el prototipo neutraliza el 100% de las corridas de ataque del conjunto de calibración con una pérdida mediana de 0 archivos (0 KiB) antes del freno, un RTO de rollback del orden de 1 s y una degradación de E/S atribuible a la capa de intercepción FUSE que se documenta por workload.

Palabras Clave—ransomware, FUSE, ZFS, snapshots, detección de comportamiento, machine learning, métricas de recuperación, RTO, falsos positivos.

I. INTRODUCCIÓN

El presente documento constituye el Informe de Avance 3, conforme al contenido propuesto en el plan del proyecto: el prototipo inicial de los módulos de detección y recuperación de ataques, luego del conjunto base de pruebas. Sobre lo presentado en el Informe de Avance 2 (implementación avanzada de la herramienta y desarrollo del modelo con su metodología de evaluación), en esta etapa se: (1) completó la campaña de recolección y el entrenamiento definitivo del modelo; (2) maduró la integración del prototipo, corrigiendo los defectos que la campaña de pruebas expuso en la capa de intercepción; y (3) ejecutó la campaña experimental que cuantifica las variables dependientes del diseño (pérdida de información, RTO, sobrecarga de almacenamiento y degradación de E/S) sobre los tres niveles de la variable independiente.

La organización sigue la del informe anterior: la Sección 2 resume el estado respecto del cronograma; la Sección 3 la madurez del prototipo; la Sección 4 el modelo entrenado; la Sección 5 la campaña experimental con sus resultados; y la Sección 6 los próximos pasos hacia el informe final.

II. ESTADO DE AVANCE RESPECTO DEL CRONOGRAMA

TABLA I. ESTADO DE AVANCE RESPECTO DEL CRONOGRAMA

Etapa (cronograma)	Tareas principales	Est
Mes 3 — Integración de modelo de ML	Relevamiento, infraestructura, desarrollo e implementación avanzada de la herramienta.	Completado (Informes 1
	Campaña de recolección (6 tandas, 135 rondas de ataque y 145 benignas) y entrenamiento definitivo.	Completado
	Conjunto base extendido (61 tests / 319 aserciones), campaña de detección (6 perfiles × 3 corridas), comparativa de escenarios, métricas de la PoC y campaña de falsos positivos. Fase de destrucción con muestras reales: pendiente.	En curso avanzado
	Métricas comparativas de la Sección 5; procesamiento estadístico final: pendiente.	Iniciado

Síntesis: el prototipo de ambos módulos (detección y recuperación) opera de extremo a extremo sobre el conjunto base de pruebas; resta la fase de destrucción con muestras reales de las familias de referencia y el análisis estadístico final.

III. MADUREZ DEL PROTOTIPO DE DETECCIÓN Y RECUPERACIÓN

La integración continua sobre la VM de laboratorio expuso y permitió corregir tres defectos de la capa de intercepción que afectaban la operatividad del prototipo como filesystem real. Cada corrección incorporó su test de regresión: la suite pasó de 28 pruebas (Informe 1) a 61 pruebas y 319 aserciones.

A. Operaciones passthrough completas

Faltaban diez handlers (chmod, chown, utimens, rmdir, statfs, access, readlink, symlink, fsync, flush): el kernel respondía ENOSYS y herramientas cotidianas fallaban dentro del punto de montaje (mkdir -p, cp, tar, rsync) con “Function not implemented”. Además, el daemon aplicaba su propio umask por sobre el del cliente, con lo que los permisos resultantes diferían de los pedidos y disparaban correcciones vía chmod. Con los handlers como passthrough directo y umask(0) al arranque, el punto de montaje se comporta como un filesystem completo.

B. Atributos completos en el listado de directorios

El listado pasaba al kernel atributos parciales (solo el tipo de entrada), que el kernel almacenaba en caché durante el timeout de atributos: ls mostraba permisos 000 y fecha 1970, y rsync replicaba esos atributos rotos al copiar desde el punto de montaje (directorios con permiso 000, imposibles de eliminar). Cada entrada se entrega ahora con su stat completo.

C. Retención de ventanas ante fallos de conexión

Si el envío de una ventana de telemetría al servidor ML fallaba (reinicio del intermediario de recolección, servidor caído), la ventana se descartaba: se perdían precisamente las ventanas de los atacantes recién neutralizados, las más valiosas. Ahora las ventanas se retienen y reintentan con la conexión restaurada (tope de 30 s), con lo que cada ataque

aporta a la telemetría todas sus ventanas (~3 por corrida, contra ~1 antes).

D. *Módulo de recuperación*

El módulo quedó validado de extremo a extremo: snapshots automáticos cada 60 s (retención 20), snapshot de emergencia asíncrono en el primer veredicto de sospecha y en cada bloqueo (retención 10), y rollback sobre el dataset real con el ataque contenido (Sección 5.C).

IV. MODELO DE MACHINE LEARNING ENTRENADO (CIERRE DEL MES 5)

A. *Campaña de recolección*

Se ejecutaron seis tandas de la campaña automatizada en modo shadow: 135 rondas de ataque y 145 benignas, produciendo un dataset de 1805 ventanas de comportamiento (1541 benignas / 264 de ataque) en 127 sesiones etiquetadas.

B. *Clases benignas difíciles*

El primer entrenamiento expuso dos falsos positivos reales que no eran defectos del algoritmo sino del dataset: (1) los escritores de bloques idénticos (heartbeats, páginas de bases de datos) eran marcados como ataque porque la constancia temporal de la entropía solo aparecía, en los datos, en los ataques; y (2) la compresión de archivos grandes producía entropía cercana al máximo sin renombrados — clase inexistente en el dataset, cuya causa raíz era que el gzip del corpus comprimible generaba una única escritura y jamás una ventana. Las rondas benignas se ampliaron con compilación, git, rsync, appends paralelos, bloques idénticos, archivos de ceros y compresión de datos incompresibles (base64 de urandom, que simula archivar medios ya comprimidos).

C. *Resultados con la metodología sin fuga*

TABLA II. MÉTRICAS DEL MODELO (VALIDACIÓN CRUZADA AGRUPADA POR SESIÓN Y HOLDOUT AGRUPADO)

	ROC-AUC (CV)	ROC-AUC (holdout)	FPR
	1.0000 ± 0.0000	1.0000	0.0000
	1.0000 ± 0.0000	0.9997	0.0000

La verificación sobre datos no vistos clasifica correctamente 80/80 ventanas benignas y 80/80 de ataque, incluidas las 50 ventanas reales de compresión (entropía > 7). Los falsos positivos de las clases difíciles pasaron de probabilidades de ataque 0.76 y 0.94 a 0.22 y 0.43 (veredicto normal).

Letra chica: el dataset se genera con el simulador de la fase de calibración; las métricas validan el circuito completo (recolección, entrenamiento, inferencia y confirmación al detector) y el aprendizaje de las clases difíciles, no la eficacia contra muestras reales, que corresponde a la fase de destrucción.

V. CAMPAÑA EXPERIMENTAL: MÉTRICAS DE LA PoC

A. *Metodología*

Entorno: VM de laboratorio aislada (Ubuntu, 4 vCPU, 3.8 GiB), pool ZFS tank/data de 2 GiB con compresión lz4, punto de montaje protegido /mnt/protected en modo de aplicación (enforce) y servidor ML activo. Las mediciones de tiempo se ejecutaron con cachés vaciadas y sin carga concurrente.

Diseño: (1) comparativa de escenarios sobre la variable independiente del proyecto — sin protección (ext4, sin snapshots), ZFS solo (dataset directo con snapshot previo y trabajo legítimo posterior) y ZFS + Guardian FS — con el mismo ataque (modo full, 40 archivos, semilla fija); (2) tasa de neutralización y pérdida por perfil de ataque (6 perfiles × 3 corridas); (3) degradación de E/S de cargas benignas (escritura secuencial de 40 MiB, 300 archivos de 64 KiB y lectura de 40 MiB; mediana de 5 repeticiones, datos incompresibles de entropía intermedia que no disparan el freno); (4) sobrecostos de almacenamiento de los snapshots; y (5) falsos positivos sobre cargas de trabajo benignas representativas.

B. *Comparativa de escenarios*

TABLA III. MISMO ATAQUE (FULL, 40 ARCHIVOS) BAJO TRES CONFIGURACIONES

Duración (s)	Archivos cifrados	RTO (s)	P
	40	n/a	40
	40	0.05	20
	1	0.03	1

La duración en los escenarios sin detección es el tiempo de cifrado completo del corpus; en el escenario con detección activa es el tiempo hasta la neutralización. La pérdida final del escenario ZFS solo corresponde al trabajo legítimo realizado desde el último snapshot (punto de recuperación); en el escenario con detección activa el snapshot de emergencia se toma en el primer veredicto de sospecha, antes del daño mayor, y el rollback restaura el corpus completo.

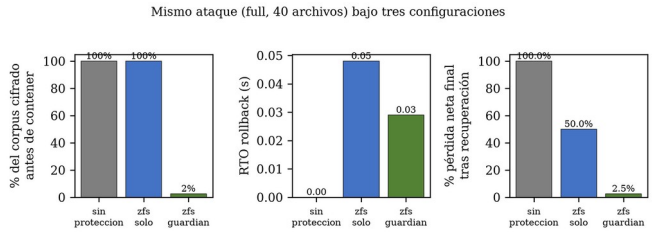


Fig. 1. Comparativa de los tres escenarios: proporción cifrada antes de contener, RTO y pérdida final tras recuperación.

C. *Neutralización y pérdida por perfil de ataque*

TABLA IV. CAMPAÑA DE DETECCIÓN POR PERFIL (3 CORRIDAS C/U, MODO ENFORCE)

Neutralizadas	Archivos .locked (mediana)	KiB cifrados (mediana)	T
	0 (máx 1)	0	0.1
	0 (máx 1)	0	0.1
	0 (máx 1)	0	2.5
	0 (máx 1)	0	1.3
	0 (máx 1)	0	0.9
	0 (máx 0)	0	—

Resultados globales: 18 de 18 corridas neutralizadas (100%), con una pérdida mediana de 0 archivos y 0 KiB cifrados antes del freno (máximo observado: 1 archivos, 102 KiB), muy por debajo del objetivo de la PoC de 1 MiB. La neutralización ocurre en el orden del segundo (la mediana incluye el arranque del intérprete del simulador).

† En el perfil multiproceso, Guardian FS termina a los cuatro workers dentro de la primera ventana (eventos process_killed con la firma de cifrado, 0 archivos cifrados), pero el orquestador del simulador queda bloqueado esperando a sus procesos muertos: el tiempo reportado para los demás perfiles no es comparable y un watchdog de la campaña lo desatasca. El comportamiento se registra como hallazgo: la neutralización parcial inmoviliza a la herramienta de ataque.

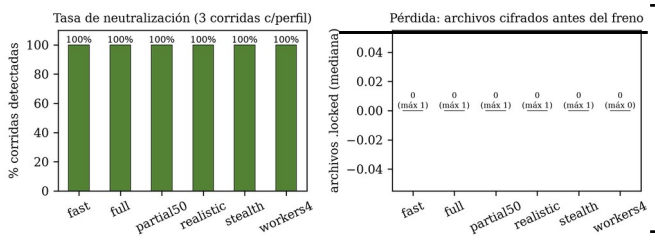


Fig. 2. Tasa de neutralización y archivos cifrados antes del freno, por perfil de ataque.

D. *Degradación de E/S en cargas benignas*

TABLA V. TIEMPO MEDIANO (MS) DE CARGAS BENIGNAS: DIRECTO ZFS VS. GUARDIAN FS

Directo (ms)	Guardian FS (ms)
524	771
201	430
267	325

El costo dominante es el análisis estadístico por byte de cada escritura (entropía y χ^2 sobre el buffer entrante, O(n)): la escritura secuencial grande lo paga íntegro (+{S['ov']['escritura_grande']}%), las operaciones pequeñas agregan el costo fijo por operación de FUSE (+{S['ov']['archivos_chicos']}%) y la lectura, que solo paga la copia del passthrough, es la carga menos penalizada (+{S['ov']['lectura']}%). El motor de detección no bloquea la syscall: el veredicto se emite dentro del handler y la inferencia ML es asíncrona. La optimización natural (muestrear el análisis a bloques representativos en escrituras grandes) queda registrada como trabajo futuro.

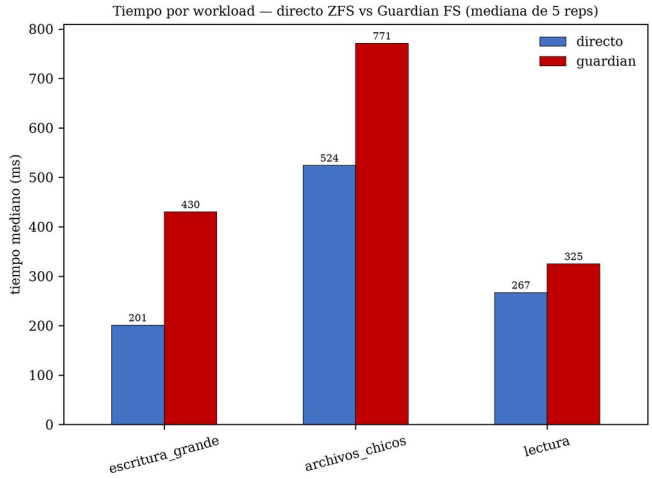


Fig. 3. Tiempo por carga de trabajo, directo ZFS vs. Guardian FS (mediana de 5 repeticiones, cachés vaciadas).

E. *Sobrecarga de almacenamiento de los snapshots*

Con el dataset de la campaña (62.3 MB referidos) y 1 snapshot retenido (guardian_auto_20261105T023752Z...), el almacenamiento adicional ocupado por los snapshots es de 0.0 MB (0.0% del dataset), por debajo del objetivo del 5% de la PoC: el Copy-on-Write de ZFS solo registra los bloques modificados desde cada foto, y la retención acotada (20 automáticos y 10 de emergencia) limita la acumulación.

F. *Falsos positivos sobre cargas benignas*

TABLA VI. CAMPAÑA DE FALSOS POSITIVOS DEL DETECTOR (MODO ENFORCE)

Bloqueos	Procesos muertos	
0	0	sí
0	0	sí
0	0	sí
0	0	sí
1	1	NO
0	0	sí
1	1	NO
0	0	sí

Dos de las ocho cargas benignas fueron terminadas (git y rsync), ambas por la misma causa raíz: el patrón de escritura atómica de archivos temporales con sufijos (.git/config.lock → .git/config; archivo.c.XXXXXX → archivo.c) que el detector síncrono interpreta como cambio de extensión, la señal clásica del ransomware. El resto de las cargas —copias de árboles, empaquetado, compilación, compresión de 40 MiB de alta entropía, appends y lectura— completó sin interferencia. El modelo de ML clasifica estas ventanas como normales; la corrección (ignorar sufijos temporales conocidos o umbral de renombrados por usuario) queda registrada como pendiente para el informe final.

VI. PRÓXIMOS PASOS

- 1. Fase de destrucción (Mes 6): ejecución de las muestras reales de las familias de referencia (LockBit 3.0, Cl0p, RansomEXX) en la VM aislada, y verificación empírica del vector de evasión mmap, con la contingencia definida en el plan (degradar FUSE a un modo asíncrono de auditoría si el bypass se confirma).
- 2. Calibración de los umbrales y pesos del detector síncrono a partir del dataset y la importancia de features del modelo entrenado (hoy calibrados manualmente).
- 3. Corrección del falso positivo residual del detector síncrono con rsync masivo (umbral de renombrados por usuario o reconocimiento de patrones de archivos temporales).
- 4. Validación del freno global con ataques multiproceso y umbral por usuario en sistemas multiusuario.
- 5. Informe final: procesamiento estadístico de la campaña, redacción y preparación de la defensa.

REFERENCIAS

[1] V. Dakic, M. Kovac, and I. Videc, “High-performance computing storage performance and design patterns—Btrfs and ZFS performance for different use cases,” Computers, vol. 13, no. 6, p. 139, 2024, doi: 10.3390/computers13060139.

[2] S. Lee et al., “Rcryptect: Real-time detection of cryptographic function in the user-space filesystem,” Computers & Security, vol. 112, p. 102512, 2021, doi: 10.1016/j.cose.2021.102512.

[3] P. O'Kane, S. Sezer, and D. Carlin, “Evolution of ransomware,” IET Networks, vol. 7, no. 5, pp. 321–327, 2018, doi: 10.1049/iet-net.2017.0207.

[4] C. Beaman et al., “Ransomware: Recent advances, analysis, challenges and future research directions,” Computers & Security, vol. 111, p. 102490, 2021, doi: 10.1016/j.cose.2021.102490.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” JAIR, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

[6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” JMLR, vol. 12, pp. 2825–2830, 2011.